

# Software Development Lifecycle

---

## Phase 1: Requirement Engineering (Defining What to Build)

This initial phase is the most critical for a project's success. Its goal is to understand and document the project's goals and constraints.

1. **Define the Problem Statement:** Prepare an initial, high-level overview of the problem to be solved, the existing system (if any), and the broad expectations for the new software. This is the activity you completed in **Experiment 1A & 1B**.
2. **Conduct a Feasibility Study:** Determine if the project is practical and worth pursuing.
  - **Technical Feasibility:** Assess if the required technology and tools can be developed or acquired.
  - **Economic Feasibility:** Analyze if the project is cost-effective, with benefits outweighing the costs.
  - **Operational Feasibility:** Determine if the new system will work within the organization's current processes and if users will accept it.
  - **Legal Feasibility:** Ensure the project complies with all relevant laws and regulations.
  - **Schedule Feasibility:** Assess if the project can realistically be completed on time.
3. **Elicit (Gather) Requirements:** Collect detailed requirements from all stakeholders (clients, users, managers). During this step, you create diagrams to capture the system's functionality from a user's perspective.
  - **Create a Use Case Diagram (Experiment 5A & 5B):** This diagram is essential for defining the system's functional requirements. It identifies the users (actors) and the actions they can perform (use cases), establishing the project's scope.
4. **Analyze Requirements:** Refine and scrutinize the gathered requirements to ensure they are consistent, unambiguous, and complete. You create several diagrams here to model the requirements and ensure a clear understanding.
  - **Model the Process Logic:**
    - **Flow Chart (Experiment 2A & 2B):** Used to visually map out the step-by-step workflow of key processes, clarifying the logic required.
  - **Model the System's Data:**
    - **ER Diagram (Entity-Relationship Diagram) (Experiment 3A & 3B):** Created to define the data the system needs to manage, identifying entities (e.g., Student, Book), their attributes, and their relationships. This forms the foundation for the database design.
  - **Model the Data Movement:**

- **Data Flow Diagram (DFD) (Experiment 4A & 4B):** Developed to illustrate how data moves between external entities, processes, and data stores within the system.

5. **Document Requirements in an SRS:** Formalize all findings and diagrams in a **Software Requirements Specification (SRS)** document. This serves as the official agreement between the development team and stakeholders.

6. **Validate and Manage Requirements:**

- **Review the SRS:** Check the document and all its diagrams for correctness, completeness, and consistency.
- **Establish Requirements Management:** Implement a process for handling any changes to the requirements that may occur during the project's lifecycle.

## Phase 2: Software Project Planning

Once the requirements are defined, this phase focuses on estimating the necessary time, effort, and resources.

- 7. **Estimate Software Size:** Predict the overall size of the project using metrics like Lines of Code (LOC) or Function Point (FP) Analysis.
- 8. **Estimate Project Cost and Effort:** Predict the cost and human effort required using models like COCOMO.
- 9. **Conduct Risk Management:** Identify, analyze, and plan for potential threats to the project.

## Phase 3: Software Design

This phase translates the "what" from the SRS into the "how" of building the software, creating a blueprint for developers. The output is a **Software Design Document (SDD)**, which includes all the following diagrams.

- 10. **Design the System Architecture (High-Level Design):** Define the overall structure and organization of the software.
  - **Component Diagram (Experiment 9A & 9B):** Shows how the system is organized into modular components (e.g., User Interface, Database, Authentication Service) and the dependencies between them.
  - **Deployment Diagram (Experiment 10A & 10B):** Visualizes the physical hardware (nodes like servers and client devices) on which the software components will be installed and run.
- 11. **Design Individual Modules (Low-Level Design):** Detail the internal structure and behavior of each component.
  - **Class Diagram (Experiment 6A & 6B):** This is the core blueprint for coding. It defines the classes, their attributes, methods, and the static relationships between them.
  - **Object Diagram (Experiment 6A & 6B):** Created to show a snapshot of objects and their relationships at a specific point in time, helping to validate the Class Diagram.

- **Sequence Diagram (Experiment 8A & 8B):** Models the dynamic interactions between objects, showing the sequence of messages exchanged to complete a specific task (like a use case).
- **Collaboration Diagram (Experiment 8A & 8B):** Also shows object interactions but emphasizes the links and relationships between objects rather than the timeline.
- **Activity Diagram (Experiment 7A & 7B):** Details the workflow of complex operations or use cases, illustrating the flow of control between various activities and decisions.
- **State-chart Diagram (Experiment 7A & 7B):** Models the lifecycle of a single, important object, showing the different states it can be in and the events that cause it to transition from one state to another.

12. **Document the Design:** Record all design decisions and diagrams in the **Software Design Document (SDD)** to guide the implementation.

## **Phase 4: Implementation and Coding**

This is the phase where developers write the actual code based on the design documents.

13. **Write the Code:** Using the chosen programming language, developers build the software components as specified in the SDD.
14. **Perform Unit Testing:** As code is written, developers test each individual function in isolation to ensure it works correctly.
15. **Use Version Control:** Employ a system like Git to track code changes and manage different software versions.

## **Phase 5: Software Testing**

This phase focuses on finding and fixing defects to ensure the software is high quality and meets all requirements.

16. **Perform Integration Testing:** Combine individual modules and test them as a group to find errors in their interactions.
17. **Perform System Testing:** Test the entire, fully integrated software to verify it meets all requirements from the SRS.
18. **Conduct Functional (Black-Box) and Structural (White-Box) Testing:** Verify the software's functionality and internal logic.
19. **Perform Acceptance Testing:** The client or end-users test the software to confirm it meets their needs.
20. **Execute Regression Testing:** After any bug fix, re-run tests to ensure that the changes have not introduced new defects.

## **Phase 6: Deployment**

Once the software has been thoroughly tested and approved, it is released to users.

21. **Plan the Release:** Create a detailed deployment plan, including server configuration and data migration.
22. **Install and Configure the Software:** Deploy the software to the production environment.
23. **Monitor the System Post-Deployment:** Closely monitor the software's performance and stability in the live environment.